
Subject: PORTING AND PARALLELISING SOLPS ON HECTOR

Author(s): Dr Fiona J. L. Reid

Distribution: Public

Porting and parallelising the SOLPS fusion code on the Cray XT6 system, HECToR

Introduction

These notes describe the porting and parallelising of the SOLPS 5.0 code on the HECToR Cray XT6 system.

SOLPS

SOLPS stands for Scrape-Off Layer Plasma Simulation and consists of a suite of codes comprising a grid generator, CARRE, a tool for specifying the material structures and providing inputs to the other codes, DG, the plasma fluid code, B2, the kinetic neutrals Monte Carlo code, EIRENE, plus plotting programs and various supporting scripts. When one or more of the codes are used together they can be used to simulate the edge plasma of tokomaks. A number of different versions of the suite currently exist and are summarised in Table 1. Please see [1] for further details.

Version	Description
SOLPS 4.0	Old version of the code which is no longer updated and is essentially considered to be complete
SOLPS 5.0	The main production trunk of the code, most regular users would use this version
SOLPS 5.1	Development version of the code where the main emphasis is on improving the EIRENE part of the code. This version allows B2 to be coupled to a parallel (MPI) version of EIRENE
SOLPS 5.2	Development version of the code where the main emphasis is on improving the B2 numerical parts, in particular with respect to the treatment of drifts and currents
SOLPS 6.0	Development version where the main emphasis is on improving the B2 numerical parts, in particular with respect to mesh adaptation

Table 1: Description of the different SOLPS versions currently available.

Following our initial discussions with the SOLPS developers it was decided to investigate the possibility of optimising and possibly parallelising the B2 component of SOLPS 5.0. The B2 component can be run either in standalone mode or it can be coupled with a kinetic neutrals code (EIRENE). As our investigations will focus on the performance of the B2 component we have mostly used B2 in standalone (i.e. without coupling to EIRENE) mode. However the code has been compiled up with EIRENE so that we can ensure that the EIRENE functionality is not affected by any changes made to the code.

Two different versions of SOLPS 5.0 have been investigated during this work. The versions were, the ÅBO version and the EUFORIA BASE version from the SOLPS SVN. A brief summary of each version is given below.

ÅBO version of SOLPS 5.0

This is a version of SOLPS 5.0 which was worked on under the EUFORIA project by the ÅBO group. This version has had calls to the NAG routines removed and the setup scripts have been tidied up such that a single script is used instead of the usual series of scripts. This version contains all components (i.e. CARRE, DG, B2, EIRENE) of the code however we have only worked with the B2 and EIRENE components.

EUFORIA BASE version of SOLPS 5.0

This is a simplified version of the main SOLPS 5.0 BASE (trunk) branch with additional code which allows a large, 92, species benchmark to be run. Unlike the main trunk, this version only contains the EIRENE and B2 components of the code. This version is obtained and managed via the SOLPS SVN allowing code changes to be fed directly back to the developers. This version became available in November 2010.

Our initial work and parallelisation was carried out using the ÅBO version of the code as this was already known to compile and run on a Cray XT system. Our latter investigations and performance analysis were carried out using the EUFORIA BASE version (once available). This version enabled us to investigate a larger problem size and also allowed the SOLPS developers direct access to the changes made to the code.

Compiling SOLPS on the HECToR Cray XT6

This section describes how to compile the SOLPS code on the HECToR Cray XT6 system. We have compiled the coupled version of the code which requires the EIRENE code to be compiled first as this is then linked to the main the SOLPS code, B2.

Libraries required by SOLPS

To compile SOLPS 5.0 the following libraries must be available:

NCARG – NCAR Graphics library

NetCDF – Network Common Data Form I/O library for reading/writing netCDF files

BLAS and LAPACK (ACML can also be used here) – linear algebra libraries

X11 – Libraries for displaying graphics, required by some components of B2 but not necessarily by EIRENE.

Compiling the ÅBO version of SOLPS 5.0 on the XT6

The procedure for compiling the ÅBO version of the code is as follows:-

1. Load the required modules:

```
module load netcdf/3.6.2
module load acml/4.2.0
module load xt-ncarg-pgi
module load xtpe-mcl2
```

2. Modify the setup.csh script such that:

```
setenv NAG "-lacml"
setenv NCARG_ROOT
setenv NCARG '-lncarg -lncarg_gks -lncarg_c -lX11 -lm'
```

The full paths to NCARG, netCDF etc are not required on HECToR as the Cray compiler wrapper scripts (ftn, cc, CC etc) handle the library paths for you.

3. Change to c-shell and run the setup.csh script to set up all the paths etc prior to the build. This step just sets up the paths, you still need to create the appropriate system specific makefiles for the actual build.

```
csh
source setup.csh linux.pgi.xt6
```

4. Now create the system specific makefiles for your system for both EIRENE and B2. For EIRENE you need to do the following

```
cd src/Eirene/config
```

Create a new file with name `compiler.os.compilername.machine`, e.g. for the XT6 machine the file, `compiler.linux.pgi.xt6` was created. The existing `compiler.linux.pgf90` file was used as a starting point and the file modified to use suit the HECToR XT6 configuration. The main entries which were altered are listed below:

```
DEFINES = ${SOLPS_DEFINES} ${EIRENE_DEFINES} \
-DRECL_UNITS_PER_DOUBLE=8 -DNO_FORT10 -DNO_SHORT_CYCLE -DNO_SAVE
FC = ftn
FFLAGS = -g -C
FFLAGS = -O3
LDLIBS = -L$(SOLPSTOP)/src/lib/$(OBJECTCODE)
```

The full listing for `compiler.linux.pgi.xt6` is given in the Appendix. If you want to create a separate version of the code for debugging purposes the simplest method is to use the `SOLPS_DEBUG ifdef` and then create a soft link to your system specific file as follows:

```
ln -s compiler.linux.pgi.xt6 compiler.linux.pgi.xt6.debug
```

For B2 a similar system specific file must be created prior to compiling the code. This file must be created in the `src/Braams/b2/base/config/` directory. The full listing for `compiler.linux.pgi.xt6` is also given in the Appendix.

5. Now build EIRENE, remembering to do a `make clean` if building from a previous installation.

```
cd src/Eirene
make > make_eirene.linux.pgi.out
```

The build should complete successfully. All the object files, module files etc get placed in the `linux.pgi.xt6/` directory. Before proceeding to build B2 you should check that you have both the `libeirene.a` and `grrlib.o` files in the `linux.pgi.xt6/` directory.

6. After building EIRENE you can then build B2 as follows:

```
cd src/Braams/b2/base
make > make_b2.linux.pgi.out
```

Again, all the object and module files relating to the build get placed in the `linux.pgi.xt6/` directory. If code builds successfully then a binary file called `b2mn.exe` will be created in this directory. E.g.

```
ls -lt linux.pgi.xt6/b2mn.exe
-rwxr-xr-x 1 fiona z01 12278943 Nov 15 10:38 linux.pgi.xt6/b2mn.exe
```

Compiling the EUFORIA BASE SVN version of SOLPS 5.0 on the XT6

The procedure for compiling the EUFORIA BASE SVN version of SOLPS 5.0 on the XT6 is as follows:-

1. Create a new directory where you'll compile this version:

```
mkdir SOLPS_SVN/
cd SOLPS_SVN/
```

2. Download the source code from the SVN repository (remember to load the SVN module first):

```
module load svn
svn co
https://solps-mdsplus.aug.ipp.mpg.de/repos/SOLPS/branches/EUFORIA/BASE/solps5.0
```

3. Load the required modules on HECToR:

```
module load netcdf/3.6.2
module load acml/4.2.0
module load xtpe-mc12
module swap pgi/10.3.0 pgi/10.6.0
module load xt-ncarg-pgi/5.1.1
```

4. Create a new set of setup files for your architecture:

```
cd solps5.0
cd SETUP
mkdir linux.pgi.xt6
cp linux_pg90_GW/* linux.pgi.xt6/.
cd linux.pgi.xt6
```

Edit the four setup files to ensure the paths to the different libraries are correct and also rename the instances of `pgf90_GW` to `linux.pgi.xt6`.

5. Create soft links to the setup files for the XT6

```
cd ../../
ln -s SETUP/linux.pgi.xt6/* .
```

6. Edit the `setup.csh.NAG.guess` file to replace the `-lblas -llapack` entries with `-lacml` to ensure the ACML math library is used. The ACML library should be faster than the BLAS/LAPACK libraries on HECToR.
7. Run the setup script to ensure the paths etc are setup correctly for compiling SOLPS. The setup script is written in c-shell and thus we need to change into c-shell first:

```
csh
source setup.csh
```

8. Create the architecture specific binaries for both EIRENE and B2 – the procedure for doing this is described under step 4 of the ÅBO build. Examples of the `compiler.*` files used for EIRENE and B2 can be found in the Appendix.

9. Build EIRENE

```
sei
Before compiling first edit the top level Makefile to ensure the system
version of makedepend is used (which makedepend will provide the path)
make listobj > make_eirene_xt6_serial.txt
make depend >> make_eirene_xt6_serial.txt
make >> make_eirene_xt6_serial.txt
```

The build will fail when attempting to build the `libobj` and `libobjx` binaries. These errors can be ignored as only the files `libeirene.a` and `grlib.o` are required to build the B2 component of the code.

10. Build B2

```
sbb
Again before compiling we need to edit the top level Makefile to ensure the
system version of makedepend is used.
make listobj > make_b2_xt6_serial.txt
make depend >> make_b2_xt6_serial.txt
make >> make_b2_xt6_serial.txt
```

Assuming no problems have been encountered then the build should complete successfully creating an executable called `b2mn.exe`. More detailed instructions on compiling SOLPS can be found at [2] and [3].

Running SOLPS

The SOLPS SVN repository contains a number of examples which can be run with SOLPS. The examples are found in the `SOLPS5.0/examples/` tree and can be downloaded individually or en-masse.

After downloading the example files the script, `b2run`, can be used to invoke SOLPS in either standalone or coupled modes. To run the `AUG_16151_D` example the following steps are required:-

1. Take a copy of the input data from the vanilla directory and place in a new `*.test/` directory

```
rsync -avP --exclude .svn 16151_1.6MW_2.0e19_D=0.4_chi=1.6_standalone/.
16151_1.6MW_2.0e19_D=0.4_chi=1.6_standalone.test/
```
2. Touch the appropriate files before new run

```
cd baserun
```

```
touch b2ag.prt b2fgmtry ; sleep 1 ; touch b2ah.prt b2fpardf ; sleep 1 ;
touch b2ai.prt b2fstati ; sleep 1 ; touch b2ar.prt b2frates
```

3. Change to run directory and touch b2fstati file

```
cd ../16151_1.6MW_2.0e19_D=0.4_chi=1.6_standalone.test/
touch b2fstati
```

4. Remove any old output files prior to run. In particular if the .prt files are not removed the run will fail.

```
rm b2mn.prt b2fstate b2fplasm* run.log *.nc ds? gnuplot* *.ps b2md.log
b2md.prt shotnumber.history*
```

5. Run SOLPS 5.0 in standalone mode using the b2run script

```
b2run b2mn > run.log &
```

The output can be compared against the sample output by using the `resall_D` script which plots the residuals using Gnuplot. Invariably a number of runs are required in order to obtain a stable solution. To continue the run from where it finished the `b2fstate` file must be copied to `b2fstati` and the run restarted e.g.

```
cp b2fstate b2fstati
b2run b2mn > run.log&
```

Figure 1 shows the provided sample results along with the results obtained from the XT6 after one 1000 step run and also for six runs (i.e one run plus 5 restarts of that run). Comparing the top and bottom graphs on Figure 1 we can see that after 5 restarts the simulation has settled down considerably and is very similar to the sample results.

With the standalone benchmark each thousand step run takes between 4-6 minutes to complete, equating to 0.22-0.24 seconds per step. The coupled version of the benchmark takes considerably longer to run (around 61 seconds per time step) and as such comparison of this type have not been carried out for the coupled case.

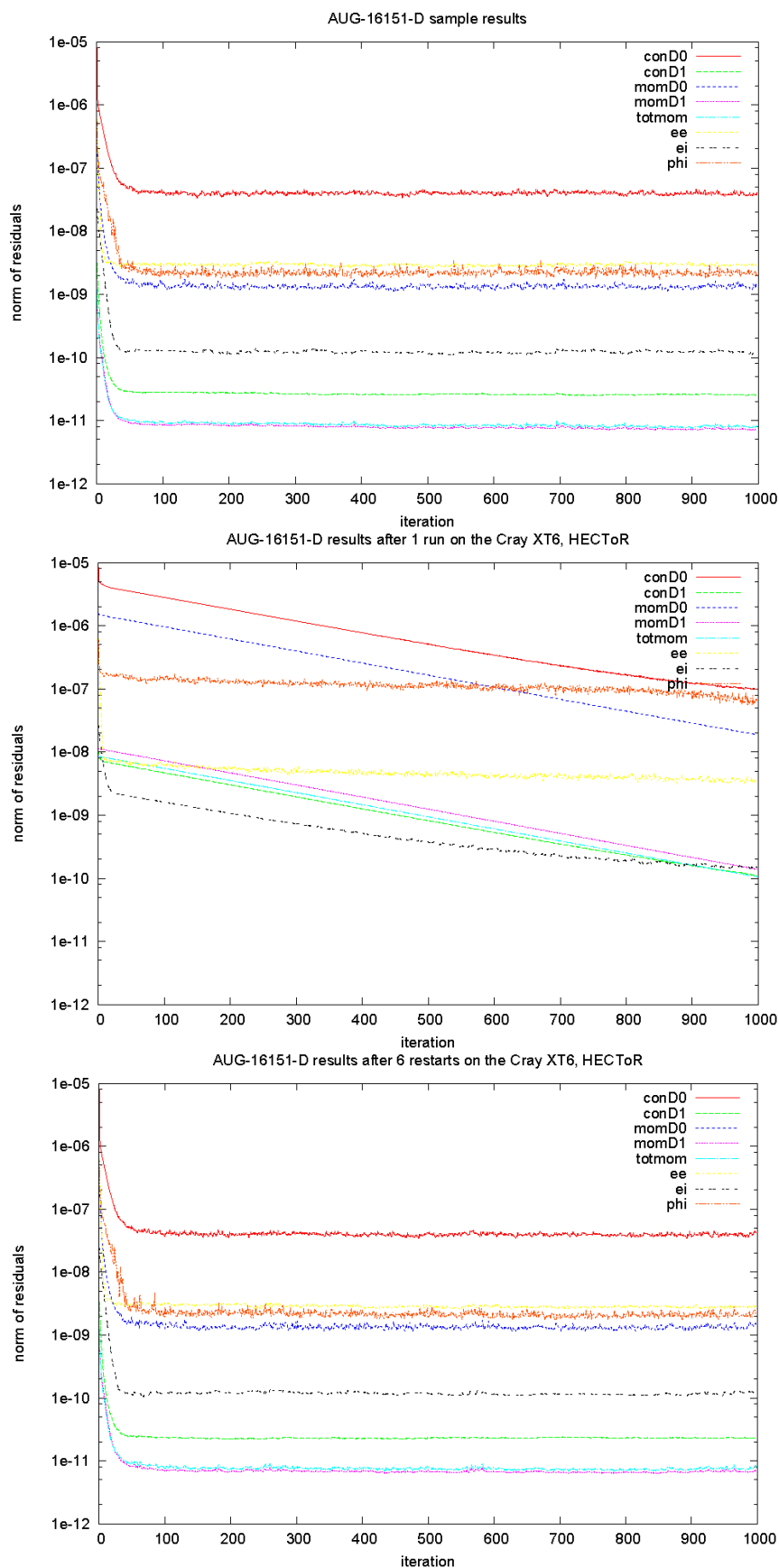


Figure 1: Showing the residuals for the AUG_16151_D benchmark. The top image shows the sample data, the middle image shows the HECToR XT6 results after 1 run and the bottom image shows the HECToR XT6 results after 5 further restarts.

Performance of SOLPS

We have run several different benchmarks using SOLPS5.0. The key details and baseline performance of the benchmarks are summarised in Table 2. The number of species corresponds to the number of different charge states, e.g. D, D+ He, He+ etc present in the benchmark.

Name	Coupled	No. of species	No. of steps run	Total runtime (seconds)	Ave time per step (seconds)	max/min	Std dev
AUG_16151_D	No	2	1000	231	0.230	0.684/0.216	0.024
AUG_16151_D	Yes	2	10	614	61.19	61.72/61.08	0.193
AUG_16151_D+C+He	No	12	1000	899	0.896	1.397/0.291	0.053
XPB tungsten	No	92	25	324	12.96	12.90/13.15	0.049

Table 2: Summarising the properties and performance of the different SOLPS 5.0 benchmarks on the Cray XT6. The XPB tungsten benchmark was only used for evaluating the parallel performance of the code.

The code reports the time taken per time step. For the AUG_16151_D benchmarks the total runtime was obtained using the Unix "time" command to find out the total runtime in seconds. For the other two benchmarks the performance timers (see page 10 for details) were used to determine the runtimes. Comparing the two standalone (non-coupled) benchmarks we see that the run time increases sub-linearly with species. For the AUG* benchmarks, the code does not appear to spend significant amounts of time start up/ shut down time (c.f. the average time per step * number of steps against the total runtime). When running the tungsten benchmark on the back end nodes significant variations in the total runtime were often observed which has been attributed to the amount of I/O that the benchmark generates (almost 1 Gbyte). As HECToR's IO is a shared resource this is to be expected.

The code has been profiled using the standard Unix `gprof` tool in order to obtain information regarding which routines are the most costly. Profiles have been obtained for the three AUG* benchmarks described in Table 2. For each benchmark only routines taking > 1% of the total run time have been listed.

The following extract shows the profile obtained for running the standalone version of the 2 species benchmark, AUG_16151_D. The benchmark was run from start up. Similar results were also obtained when restarting the run which is a more usual case for a real science user of the code.

```
Flat profile:
Each sample counts as 0.01 seconds.
```

%	cumulative	self	self	total		name
time	seconds	seconds	calls	s/call	s/call	
19.71	38.39	38.39	10998	0.00	0.00	ma30bd_
18.01	73.47	35.07	22000	0.00	0.00	ma30cd_
6.60	86.33	12.86	12000	0.00	0.00	b2tfnb_
4.53	95.16	8.83	10998	0.00	0.00	ma28dd_
4.27	103.48	8.32	11000	0.00	0.00	ma28id_
2.50	108.34	4.87	11000	0.00	0.01	ilutern_
1.95	112.14	3.80	4000	0.00	0.00	b2tfhi_
1.92	115.89	3.75				__c_mzero4
1.79	119.38	3.49	4000	0.00	0.00	b2tfch_
1.65	122.59	3.21	2	1.61	1.61	ma30ad_
1.64	125.79	3.20				__c_mset8
1.60	128.90	3.11				mstep_
1.60	132.00	3.11				__c_mset4
1.27	134.48	2.48	2001	0.00	0.00	b2stel_
1.13	136.69	2.21				__c_mzero8
1.13	138.90	2.21	2002	0.00	0.00	b2sqel_
1.11	141.06	2.16	4000	0.00	0.00	b2tfhe_

Profile obtained for the coupled version of AUG_16151_D:

Flat profile:

Each sample counts as 0.01 seconds.

%	cumulative	self		self	total	
time	seconds	seconds	calls	s/call	s/call	name
19.67	194.88	194.88	161074312	0.00	0.00	timer
16.07	354.03	159.16	40429769	0.00	0.00	timeal
12.21	475.00	120.97	161074312	0.00	0.00	timep
6.23	536.70	61.70				__rouexit
4.72	583.44	46.74	20	2.34	2.34	xstaii_
3.15	614.67	31.23				_mcount2
2.72	641.66	26.98	105813834	0.00	0.00	fpatha_
2.42	665.62	23.96	122633822	0.00	0.00	uptcop_
2.06	685.98	20.36				__rouinit
2.05	706.24	20.27				sort_
2.00	726.09	19.84	5009293	0.00	0.00	folneut_
1.55	741.40	15.31				f90io_backspace
1.41	755.35	13.96				__c_mcopy4
1.03	765.52	10.17	2998	0.00	0.00	ma30bd_

In the coupled profile the time* routines (grey shading) are part of the EIRENE code suggesting that a significant part of the coupled run is spent within EIRENE. Indeed comparing the standalone versus coupled AUG_16151_D benchmark times we can see that the addition of the EIRENE component is very costly.

The profile for the standalone 12 species benchmark, AUG_16151_D+He+C is given below:

Flat profile:

Each sample counts as 0.01 seconds.

%	cumulative	self		self	total	
time	seconds	seconds	calls	s/call	s/call	name
14.43	108.40	108.40	32082	0.00	0.00	ma30bd_
13.55	210.19	101.79	85008	0.00	0.00	b2tfnb_
12.73	305.82	95.63	64168	0.00	0.00	ma30cd_
3.24	330.12	24.30	32082	0.00	0.00	ma28dd_
3.01	352.72	22.60	32084	0.00	0.00	ma28id_
2.72	373.15	20.43	2001	0.01	0.01	b2stel_
2.06	388.60	15.45	1000	0.02	0.02	b2sihs_
1.85	402.49	13.89	2002	0.01	0.01	b2sqel_
1.67	415.02	12.53	32084	0.00	0.01	ilutern_
1.55	426.66	11.64	5084	0.00	0.00	b2tfhi_
1.38	437.06	10.40	12000	0.00	0.00	b2sifr_
1.31	446.87	9.81	2001	0.00	0.00	b2stcx_
1.20	455.85	8.98	1000	0.01	0.01	tallies_
1.17	464.66	8.81	1001	0.01	0.01	b2spel_
1.08	472.81	8.14				mstep_
1.06	480.75	7.94				__c_mzero4
1.06	488.69	7.93				__fmth_i_dlog_gh
1.05	496.56	7.87	2	3.94	3.94	ma30ad_

If we compare the two standalone profiles, the top five routines are the same in either case. For the AUG_16151_D+He+C benchmark, the b2tfnb routine becomes more costly, overtaking the ma30cd routine which suggests that b2tfnb is affected by the number of species. A brief summary of the function of the top 5 routines is given below.

- ma30bd – In source file utility/ma28.F. Performs the LU decomposition of the diagonal blocks of a matrix using information obtained from a previous call to ma30a/ad. No child routines called from here.

- `b2tfnb` – In source file `equations/b2news.F`. Subroutine `b2tfnb` computes the particle flux, `fnb`, of one atomic species between neighbouring cells, and the coefficients (`flob` and `conb`) that specify its linearization.
- `ma30cd` – In source file `utility/ma28.F`. Called from subroutine `ma28cd`, no children. The routine `ma30c/cd` uses the factors produced by `ma30a/ad` or `ma30b/bd` to solve a matrix $ax=b$ or a transpose of $x=b$ where the matrix $p1*a*q1$ (`paq`) is the block lower triangular.
- `ma28dd` – As above, found in source file `utility/ma28.F`. Called from subroutine `ma28bd`, no children. This routine sorts the matrix as given by the user into the structure of the decomposed form and checks for the presence of duplicate entries or non-zeros lying outside the sparsity pattern of the decomposition. It also calculates the largest element in the input matrix.
- `ma28id` – In source file `utility/ilutern_7_solps.F`. Called from subroutine `ilutern`, makes 2 calls to `ma28cd`.

```
subroutine ma28id(n, nz, aorg, irnorg, icnorg, licn, a, icn, &
    ikeep, rhs, x, r, w, mtype, prec, iflag)
```

The `gprof` results tell us which routines are the most costly however they do not provide any information regarding the overall structure of the code and potential parallelisation strategies. The next section describes how some additional timers were added at subroutine level enabling an improved understanding of the overall structure of the code to be obtained.

Utilising the existing `subini/ subend` calls to add timers to the code

Inside many of the B2 source files there are existing calls to `subini` and `subend` at the start and end of many of the subroutines. These calls are normally used to build up a stack trace such that when debugging the code the location of a crash can be easily pinpointed. The calls should be easily adaptable such that timers at subroutine or function level can be inserted into the code.

We have used the performance monitoring code written for the EUFORIA work on GENE as a starting point. The code, now called `perf_fiona.F`, has been modified as follows:-

- indentation 6 chars in required (fixed format Fortran)
- swap `par_mod` for `b2mod_timer`
- The two `*manual*` timing routines have been removed

A new module has been added under `modules/b2mod_timer.F` which contains the additional arrays used for storing the timer data. Changes were also made to the existing `utility/subsys.F` and `utility/sysend.F` files:

- In `subsys` – add use `b2mod_timer` and call `perfinit` to initialise the timer arrays
- In `subini` – added a call to `perfon("name_of_routine")`
- In `subend` – added a call to `perfoff`
- In `sysend` – added a call to `perfout_all` to ensure results from timers get written to standard output at the end of the normal SOLPS run.

After making these changes, SOLPS5.0 was recompiled and the results checked for correctness. The code runs as normal and the overhead due to the timing calls is negligible.

Pre-processor `#ifdef/#endif` calls have been used to allow these timers to be turned on or off by the user. To use these timing calls simply add `"-DPERF_TIMER"` to the system specific Makefile for B2. The advantage of using these timers in addition to a traditional profiler is that you obtain a higher level overview of the code structure (e.g. the hierarchy of routines) and also gain further understanding of which section of the code is most costly which may aid in finding a way of parallelising the code.

The timing results obtained from the AUG_16151_D and AUG_16151_D+C+He standalone benchmarks can be found in Table 3.

AUG_16151_D				AUG_16151_D+C+He			
Timer number	Routine name	Time(s)	Calls	Timer number	Routine name	Time(s)	Calls
1	b2mn	245.761	1	1	b2mn	907.624	1
2	b2mndr	245.572	1	2	b2mndr	907.004	1
3	b2mnds	0.016	1	3	b2mnds	0.072	1
4	b2rflb	0.000	1	4	b2rflb	0.000	1
5	b2rucp	0.000	1	5	b2rucp	0.004	1
6	b2ruzd	0.000	2	6	b2ruzd	0.000	2
7	b2xvcp	0.000	4	7	b2xvcp	0.000	4
8	b2rfcp	0.000	1	8	b2rfcp	0.000	1
9	b2rurc	0.0120	1	9	b2rurc	0.056	1
10	b2rusr	0.000	1	10	b2rusr	0.000	1
11	b2wfcg	0.000	1	11	b2wfcg	0.008	1
12	b2wucp	0.000	1	12	b2wucp	0.004	1
13	b2wuzd	0.000	4	13	b2wuzd	0.000	4
14	b2mwq0	0.000	1	14	b2mwq0	0.000	1
15	b2rugm	0.080	1	15	b2rugm	0.080	1
16	b2xvgm	0.000	2	16	b2xvgm	0.004	2
17	get_jsep	0.000	6	17	get_jsep	0.000	6
18	b2xgbs	0.000	1	18	b2xgbs	0.000	1
19	b2rupg	0.072	1	19	b2rupg	0.228	1
20	b2xvps	0.144	1025	20	b2xvps	0.444	1025
21	b2spel	2.141	1001	21	b2spel	11.578	1001
22	b2sqel	4.754	2002	22	b2sqel	26.217	2002
23	b2wfgm	0.004	1	23	b2wfgm	0.004	1
24	b2wfpi	0.000	2	24	b2wfpi	0.004	2
25	b2wfpj	0.000	2	25	b2wfpj	0.004	2
26	b2mxnp	0.168	5000	26	b2mxnp	0.388	5000
27	b2mxnu	0.544	6000	27	b2mxnu	1.125	6000
28	b2mndt	231.275	1000	28	b2mndt	870.189	1000
29	b2spcx	0.436	1000	29	b2spcx	1.213	1000
30	b2tral	6.171	1001	30	b2tral	53.223	1001
31	b2trno	2.745	1001	31	b2trno	19.698	1001
32	b2tqna	0.924	1001	32	b2tqna	5.195	1001
33	b2tlh0	0.168	1001	33	b2tlh0	8.356	1001
34	b2txcx	1.240	19019	34	b2txcx	10.573	199199
35	b2txcy	0.764	19019	35	b2txcy	7.716	199199
36	b2txsx	0.084	2002	36	b2txsx	0.648	12012
37	b2txsy	0.104	2002	37	b2txsy	0.428	12012
38	b2txfx	0.088	1001	38	b2txfx	0.292	1001
39	b2txfy	0.076	1001	39	b2txfy	0.276	1001
40	b2trcl	2.917	1001	40	b2trcl	31.996	1001
41	b2tqca	0.496	1001	41	b2tqca	3.514	1001
42	b2tqce	0.468	1001	42	b2tqce	0.432	1001
43	b2tqin	0.248	1001	43	b2tqin	9.849	1001
44	b2tlmv	0.196	2002	44	b2tlmv	0.960	12012
45	b2sral	26.653	2001	45	b2sral	122.717	2001

46	b2stbc	4.490	2001	46	b2stbc	11.789	2001
47	b2stbc_phys	2.329	2001	47	b2stbc_phys	6.663	2001
48	b2stbr	5.171	2001	48	b2stbr	22.003	2001
49	init_wall	0.012	1	49	init_wall	0.008	1
50	b2mod_wall	0.000	1	50	b2mod_wall	0.000	1
51	wall_data	0.000	1	51	wall_data	0.076	1
52	b2stbr_phys	1.505	2001	52	b2stbr_phys	15.900	2001
53	b2stbm	0.632	2001	53	b2stbm	0.888	2001
54	b2stel	4.026	2001	54	b2stel	26.197	2001
55	b2sqcx	0.600	2001	55	b2sqcx	2.373	2001
56	b2stcx	1.224	2001	56	b2stcx	13.159	2001
57	b2srsm	0.165	2001	57	b2srsm	0.444	2001
58	b2srdt	0.952	2001	58	b2srdt	2.893	2001
59	b2srst	0.724	2001	59	b2srst	3.066	2001
60	b2news	212.974	1000	60	b2news	787.564	1000
61	b2treq	0.292	1000	61	b2treq	0.304	1000
62	b2trql	0.244	1000	62	b2trql	0.344	1000
63	b2tlhe	0.076	1000	63	b2tlhe	0.060	1000
64	b2tlhi	0.124	1000	64	b2tlhi	0.084	1000
65	b2tfnb	17.145	12000	65	b2tfnb	148.638	85008
66	b2npmo	46.167	1000	66	b2npmo	210.437	1000
67	b2nxfx	0.168	1000	67	b2nxfx	0.224	1000
68	b2nxdu	0.196	1000	68	b2nxdu	0.832	1000
69	b2sifr	1.361	2000	69	b2sifr	27.178	12000
70	b2sigp	0.384	2000	70	b2sigp	2.381	12000
71	b2nxfc	0.312	2000	71	b2nxfc	1.313	12000
72	b2nxdv	0.068	2000	72	b2nxdv	0.392	12000
73	b2urmo	0.820	4000	73	b2urmo	4.730	24000
74	b2usmo	40.000	3000	74	b2usmo	159.892	13000
75	b2ux5p	138.665	11000	75	b2ux5p	373.503	32084
76	b2nxst	0.064	2000	76	b2nxst	0.452	12000
77	b2nxcn	0.120	1000	77	b2nxcn	0.604	1000
78	b2npco	26.913	2000	78	b2npco	150.015	12000
79	b2ursc	0.296	2000	79	b2ursc	1.369	12000
80	b2usco	26.281	2000	80	b2usco	146.54	12000
81	b2upco	0.280	2000	81	b2upco	1.849	12000
82	b2nxdp	0.060	1000	82	b2nxdp	0.020	1000
83	b2tfch	6.331	4000	83	b2tfch	6.979	5084
84	b2tfhe	3.518	4000	84	b2tfhe	4.066	5084
85	b2tfhi	4.858	4000	85	b2tfhi	12.977	5084
86	b2nppo	39.340	3000	86	b2nppo	49.505	4084
87	b2ursd	0.648	5000	87	b2ursd	0.696	6084
88	b2uspo	38.775	3000	88	b2uspo	48.952	4084
89	b2uppo	0.048	3000	89	b2uppo	0.044	4084
90	b2tfcc	0.304	1000	90	b2tfcc	1.909	1000
91	b2sdia	0.020	1000	91	b2sdia	0.012	1000
92	b2npht	43.054	1000	92	b2npht	64.168	1000
93	b2sihs	2.345	1000	93	b2sihs	26.201	1000
94	b2usht	39.800	1000	94	b2usht	36.846	1000
95	b2upht	0.252	1000	95	b2upht	0.464	1000
96	b2stbr_neutr_scl	0.000	1000	96	b2stbr_neutr_scl	0.000	1000
97	b2mxzr	0.008	1000	97	b2mxzr	0.060	1000
98	b2mxar	0.348	1000	98	b2mxar	1.973	1000
99	b2mxac	0.092	1000	99	b2mxac	0.356	1000
100	b2mwqt	0.080	1000	100	b2mwqt	0.136	1000
101	tallies	11.421	1000	101	b2mwti	11.658	1000
102	readuser	0.000	1	102	output_ds	0.004	4
103	b2mwti	0.884	100	103	tallies	18.941	1000
104	output_ds	0.004	4	104	readuser	0.000	1
105	b2wups	0.204	1	105	b2wups	0.700	1
106	b2wdat	0.000	1	106	b2wdat	0.000	1

Table 3: Showing output from subroutine timers for AUG_16151_D and AUG_16151_D+C+He in standalone mode.

From Table 3 we can see that the main SOLPS5.0 routine is `b2mn` with majority of the codes runtime being spent within the subroutine `b2mndr` (the main driver routine). The driver routine calls `b2mndt` (the time stepping routine) which in turn invokes `b2news`. Over 85% of the runtime is spent within `b2news` and thus if we intend to parallelise the code understanding what `b2news` does and finding a way to carry out this part of the computation in parallel will be crucial.

The `b2news` routine performs an inner iteration of the B2 equations and defines the core structure of the solution algorithm (see the code source for more details). In computing the B2 equations the following steps are carried out:

1. test input data
2. compute auxiliary quantities
3. compute transport coefficients
4. iterate (solve equations) on parallel momentum balance
5. iterate (solve equations) on particle balance
6. iterate on charge balance
7. iterate on heat balance

Steps 4 and 5 involve solving the momentum and continuity equations (where the costly `b2tfnb` and the `ma28/30*` routines are called). These equations are computed for all species for each time step.

OpenMP parallelisation of the nspecies loops

Our initial discussions with the SOLPS developers suggested that the species computations are completely independent and thus it should be possible to parallelise the code at the species level. The main loops over species (4 loops) are contained within the `b2news` routine. In addition, a further loop over species can be found within the `b2npmo` subroutine.

We will use OpenMP to investigate the feasibility of this approach. OpenMP is chosen as it is relatively quick to implement with only small modifications to the source code required. The 4 loops over species are summarised as follows with line numbers from the original version of the code given for reference.

1. Loop 1, approx lines 267-281 contains calls to `b2xppb` and `b2tfnb`
2. Loop 2, lines 314-395, contains a loop over `ny`, and `nx` (loops over particles), contains a large amount of output to standard output. The routine always calls `b2xppb` and `b2tfnb`. There are conditional statements controlling (`if solving(2)`) whether `b2npco`, `xertst` and `b2fntb` get called. This loop contains the calls to the solver for the continuity equations and therefore is likely to be the most costly.
3. Loop 3, lines 541-557, calls `b2xppb` and `b2tfnb`
4. Loop 4, lines 699-701, single line of array syntax which sums up the values of the `ue` array.
5. Loop 5 is contained within the `b2npmo` subroutine. It is similar in form to loop 2 but relates to the momentum calculation. The `b2npmo` routine contains a loop over species which computes the momentum for each species followed by a combined computation which requires the results from all species to obtain the combined solution. As a result only the initial loop over species can be sensibly be parallelised.

As loop 4 is the simplest, containing just a single line of code we have begun by converting it to use OpenMP directives. The loop contains a single line of code as shown below:

```
do is=0,ns-1 ! ns is the number of species
  ue(:, :)=ue(:, :)+rza(:, :,is)*ua(:, :,is)*na(:, :,is)
enddo
```

When converting a loop to use OpenMP we need to determine which variables can be shared between threads and which variables must be private to each thread. We must also ensure that operations which require input from all threads are carried out correctly. For example in the code snippet above, the array, `ue` is computed by adding on its current value for each species in turn. When using OpenMP each thread will work on one or more species with each computing their own contribution to `ue`. As such we must ensure that the final summation is carried out correctly by using the OpenMP REDUCTION clause which will collect the values from all threads and apply the appropriate arithmetic operation (in this case summation) before returning the final value to the master thread (thread 0) at the end of the parallel region.

After addition of OpenMP directives, loop 3 now looks like:

```
!$OMP PARALLEL DO SHARED(rza,ua,na,ns) REDUCTION(+:ue)
  do is=0,ns-1
    ue(:, :)=ue(:, :) + rza(:, :,is)*ua(:, :,is)*na(:, :,is)
  end do
!$OMP END PARALLEL DO
```

The variables `rza`, `ua`, `na` and `ns` are all accessed in a read only context and thus a single copy of these variables can be shared between all threads. However, the `ue` array needs to be declared with the reduction clause as discussed above.

Loop 1 over species contains two subroutine calls to `b2xppb` and `b2tfnb`. `b2xppb` computes the values of the `pb(:, :)` array for each species. The `b2tfnb` routine then uses the newly computed `pb(:, :)` array to compute a number of species specific values. Before applying OpenMP directives to this loop we tried reversing the order of iterations, e.g. running from `ns-1` to `-1` in steps of `-1` to ensure that the loop order was unimportant. The `flob` and `conb` arrays are also output by `b2tfnb` and thus should be declared private to each thread. It turns out that in loop 1 the computed values of `flob` and `conb` are not actually used but as potentially more than one thread could attempt to write to these values simultaneously they should still be declared as private variables. Loop 1 was modified to look like:-

```
!$OMP PARALLEL DO PRIVATE(pb, flob, conb)
  do is = 0, ns-1
*    ..compute partial pressure
    call b2xppb (nx, ny, rza(-1,-1,is), na(-1,-1,is), te, ti, pb)
*    ..compute particle flux
    call b2tfnb (nx, ny,
1      guard_flows, facdrift, fac_ExB, am(is)*mp,
1      vol, rza(-1,-1,is), pbs, na(-1,-1,is), pb, ua(-1,-1,is),
1      uadia(-1,-1,0,is), vaecrb(-1,-1,0,is), vadia(-1,-1,0,is),
&      wadia(-1,-1,0,is), ne, te, ti, bb, hx, hy, qz, gs, qc, po,
&      cvla(-1,-1,0,is), cdna(-1,-1,0,is), cdpa(-1,-1,0,is),
&      fna(-1,-1,0,is), fna_nodrift(-1,-1,0,is), flob, conb,
1      fna_32(-1,-1,0,is), fna_52(-1,-1,0,is),
1      fna_32nd(-1,-1,0,is), fna_52nd(-1,-1,0,is),
2      kinrgy(-1,-1,is), fllim0fna(-1,-1,0,is))
  enddo
!$OMP END PARALLEL DO
```

The `pb`, `conb` and `flob` arrays need to be private to each thread. All the other variables/arrays can be shared as they are either read-only or only written to based upon their species number.

In addition to determining whether variables contained within the loop body are shared or private we must also ensure that any variables accessed by host association or via the subroutine calls to `b2xppb` and `b2tfnb` are also correctly handled. In any particular variables defined within modules or local subroutine variables with the `save` attribute require special handling.

In Fortran 90/95 Module (or common block) data and `save` variable data is global data. If this type of data accessed inside a parallel region a single shared copy of this data will exist in memory. Therefore, if each thread requires its own copy of any global data the variables must be declared as `THREADPRIVATE`. In the `b2xppb` routine no such changes are required. This routine has no `save` variables and the only module it uses is `b2mod_types`, which declares some type parameters which don't change and thus can be global.

In the `b2tfnb` routine the situation is more complex. This routine uses a number of modules and also declares a number of local `save` variables whose values persist between calls to the subroutine. We shall consider the modules first and then the `save` variables.

The modules used by `b2tfnb` are: `b2mod_types`, `b2mod_boundary_namelist`, `b2mod_indirect`, `b2mod_constants`, `b2mod_geo_corner`, `b2mod_b2cmpa`, `b2mod_b2cmfs`. The variables declared in these modules are all used in a read-only context inside the parallel loop and thus each thread can access the same global variable in memory. As such, no `THREADPRIVATE` declarations are required for these modules.

The `save` variables in `b2tfnb` are: `ncall`, `alpha`, `gamma`, `flux_limit_min_ti`, `area_fix`, `no_hybr`. With the exception of `ncall`, these variables are effectively used as constants with their values either set to the default specified in `b2tfnb` or read in from an input file. The function of `ncall` is somewhat different. Within many SOLPS subroutines the `ncall` variable is used as a local counter to count the number of times a particular subroutine has been called. Depending on the value of `ncall` a number of different operations are typically performed. These can be summarised as follows:-

1. `ncall` used as a first time flag (e.g. `ncall == 0`). The first time the routine is entered a number of variables are initialised and their values saved between calls. The first time flag can also be used to allocate memory.
2. `ncall` used to test for errors (e.g. `ncall <= 3`). For the first e.g. 3 calls to the routine perform some checks on the input values to ensure that the variables, arrays etc contain sensible values.
3. `ncall` used to control output frequency (e.g. `ncall < 100` or `ncall == 1000` etc). Controls the output to screen (or file) depending on how many times the routine has been called.

Figure 2 shows pseudo-code illustrating the uses of the `ncall` variable in `b2tfnb`.

```

subroutine b2tfnb(args)
  module use statements
  variable declarations

  if (ncall == 0) then
    set internal parameters, i.e. alpha, gamma etc on the
    first call to b2tfnb
  end if

  if (ncall <= 3) then
    check sign of nb, ne, pb, cdnb & cdpb
    code will stop if errors are encountered
  end if

  compute drift velocities, fluxes etc
  compute fnbx, flobx, conbx, fnby, floby, conby

  if (certain conditions are met and ncall < 100) then
    print max flux limit to screen
  end if

  compute boundary values and kinetic energy of this species

  ncall = ncall + 1      ! update ncall
  return
end subroutine b2tfnb

```

Figure 2: Pseudo-code for the `b2tfnb` subroutine highlighting the different uses of the `ncall` variable. The grey shading indicates operation 1 (first time use), green shading indicates operation 2 (error checking use) and magenta shading indicates operation 3 (i/o frequency use).

As `ncall` is declared with the `save` attribute this means a single global copy of it exists in memory and thus when `b2tfnb` is called from inside a parallel loop all threads are able to update this variable. This could potentially lead to a race condition where one thread reads the variable before another thread has updated it and therefore the `ncall` variable must be declared as `THREADPRIVATE` so that each thread will have its own copy of the variable.

When `ncall` is used as a first time flag the code will run just as the serial version did, except each thread will now perform its own initialisation of the variables, `alpha`, `gamma` etc. As such `alpha`, `gamma` (i.e. all the remaining `save` variables in `b2tfnb`) etc must also be declared as `THREADPRIVATE`. When `ncall` is used for error checking the behaviour will be slightly different, now, each thread will perform its own 3 tests such that additional species and/or time steps will now be tested for correctness. As all the tests should pass (unless something is wrong in which case the code would report an error and stop) this should not pose any problems. Instead of a total of 3 tests being performed, each thread will now perform 3 tests and thus the number of tests will increase by a factor of `OMP_NUM_THREADS` times. However, if identical behaviour to the serial code is required this can be achieved by placing a condition on the `PARALLEL DO` in `b2news` such that the code will run in serial if `(ncall*ns < 3)` and by adding a new `save` variable to `b2tfnb` called for example, `fr_nctest`, which is only used for the first few calls to the routine.

The situation where `ncall` is used to control the output frequency is more complex. When running the code on many threads the ordering of the output will not be guaranteed. As a result, outputting data corresponding to the same value of `ncall` that would have been encountered in serial is extremely tricky especially if anything other than a static schedule in OpenMP is used. On examining the code it is

apparent that when used to control output frequency, `ncall`, is used in a somewhat arbitrary fashion and looks very much like methods of obtaining diagnostic data or for debugging. Indeed, discussions with the code developers have confirmed this and they have suggested that any I/O contained with parallel loops and affected by having a thread specific value of `ncall` could be pre-pended with a warning to let users know that the output may not be quite what they anticipate.

This has been achieved using a combination of a pre-processor macro, `_OPENMP`, along with the `OMP_IN_PARALLEL` function to ensure that the code will annotate the output appropriately. The `b2tfnb` routine can potentially be called from sequential *and* parallel parts of the code and thus all eventualities need to be accounted for. The pseudo code below illustrates the method used:

```
#ifdef _OPENMP
if (OMP_IN_PARALLEL)
  write(*,*)"OpenMP warning: Original serial output"
else
  write(*,*)"Original serial output"
end if
#else
write(*,*)"Original serial output"
#endif
```

The `OMP_IN_PARALLEL` conditional is required to ensure that only the output from inside a parallel region is pre-pended and non parallel accesses will behave as normal.

Loop 3 is identical in structure to loop 1 and therefore can be converted to use OpenMP in exactly the same way with just the `pb`, `flob` and `conb` arrays being made private to each thread.

Loop 2 is somewhat more complex and contains a loop over particles followed by a number of different subroutine calls depending on the value of `solving(2)`. The `solving` array is a one dimensional logical array containing four elements whose values control which equations are being solved for. The four elements represent respectively the momentum, continuity, potential and heat equations. The majority of loop 2 consists of local code or calls to the `b2xppb` or `b2tfnb` routines which we have already encountered. There is also an additional call to the `b2npco` subroutine which is only made if `solving(2)` is true. This `b2npco` subroutine solves the continuity equations for each species.

At first glance, the iterations of loop 2 appear to be completely independent. Indeed, reversing the order of the loop produces identical results to running the loop in the forwards direction, suggesting that an OpenMP approach should be viable for this loop. Initial attempts to use OpenMP with this loop involved making the `pb`, `flob`, `conb`, `solvedum`, `solvedm2`, `mach10` and `ier0` variables private to each thread with all other variables being shared between threads. The code ran successfully on 1 thread but failed with a segmentation violation when running on 2 or more threads suggesting that a closer look at exactly what the `b2npco` routine does is required.

It turns out that the `b2npco` routine is rather complex and involves a large number of additional subroutine calls. The call tree to the actual solver routines involves six layers of calls:-

```
b2news -> b2npco -> b2usco -> b2ux5p -> ilutern_solps -> ma28/30**
```

At each of these levels we need to ensure that any global variables (i.e. module variables, common block variables or save variables) which are not accessed in a read only context must be made local to each thread. Any global variables which are accessed in a read only manner can be safely shared between all threads. Each of the routines in the above call tree and any modules accessed via host association were examined and the appropriate variables declared as THREADPRIVATE.

In addition to the routines mentioned above a number of other routines also needed small modifications to ensure that they would work correctly in OpenMP. Most of these modifications just involve declaring the `ncall` variable to be THREADPRIVATE. A list of the altered source files and brief summary of the changes made can be found in the Appendix on page 26.

The `b2ux5p` routine required some additional changes. This routine has been modified to ensure that the code will stop if either the `iluter` or `slv5pt` solvers are used as these have not been parallelised. The logic used is similar to that used to label the OpenMP output in the `b2tfnb` routine.

After adding OpenMP directives to loop 2 it looks like:

```
!$OMP PARALLEL DO DEFAULT(none)
!$OMP& PRIVATE(pb, ier0, solvedum, solvedm2, flob, conb, mach10)
!$OMP& SHARED(ns, nx, ny, region, leftix, rightix, bottomiy)
!$OMP& SHARED(topiy, na, rza, te, ti, am, ua, ne)
!$OMP& SHARED(is_neutral, uadia, ev, ncall, ncallout, resmo, corua)
!$OMP& SHARED(guard_flows, facdrift, fac_ExB, vol, pbs, vaecrb, vadia)
!$OMP& SHARED(wadia, bb, hx, hy, qz, gs, qc, po, cvla, cdna, cdpa)
!$OMP& SHARED(fna, fna_nodrift, fna_32, fna_52, fna_32nd, fna_52nd)
!$OMP& SHARED(kinrgy, fllim0fna, solving, nnreg, solveco, solvemo)
!$OMP& SHARED(rxf0, sna, pcca, resco, corpa, b2stbr_sna)
    do is = 0, ns-1
        main body of loop in here.. calls b2xppb, b2tfnb,
        b2npco etc
    end do
!$OMP END PARALLEL DO
```

Finally, the momentum computation contained inside the `b2nmpo` routine was also parallelised in a similar way to the continuity equations in loop 2. A number of additional routines required alteration. Again, full details of the changes made to the source code can be found in the Appendix.

At each stage of the parallelisation process the code was tested against the serial results to ensure that no significant differences occurred. The serial version was also tested without any OpenMP flags to ensure that it still performs as expected for users without access to a parallel system. The final output from the residuals, e.g. obtained from the `resall_D`, `resco` or `resmo` scripts were compared against the serial results with the maximum difference being reported. The largest differences were found to be of the order 1.0E-07 and resulted from differences in calculation order that occur for example when performing reduction operations. When running on multiple threads operations involving summations of a single variable cannot be guaranteed to give the same results as when running in serial as the order of summation cannot be guaranteed. However, the differences between the parallel and serial versions of the code are small and the developers have confirmed these differences will not pose any problems.

The initial OpenMP parallelisation and testing was carried out using the ÅBO version of the code and when we were satisfied that this was working correctly the changes were merged into the main EUFORIA BASE SVN branch of the code so that the

large 92 species benchmark could be used. When merging the changes into the EUFORIA BASE version, testing as described above was carried out at each stage of the process.

The timings obtained after adding the OpenMP directives to the EUFORIA BASE version of the code are given by Table 4. The timings were obtained on the master thread (thread 0) to ensure that only one thread is able to add/subtract from the timer call stack. If thread specific timers are needed then an additional array dimension needs to be added to the main timer and counter arrays such that each thread accesses its own memory space. The four timings reported in Table 4 correspond to:

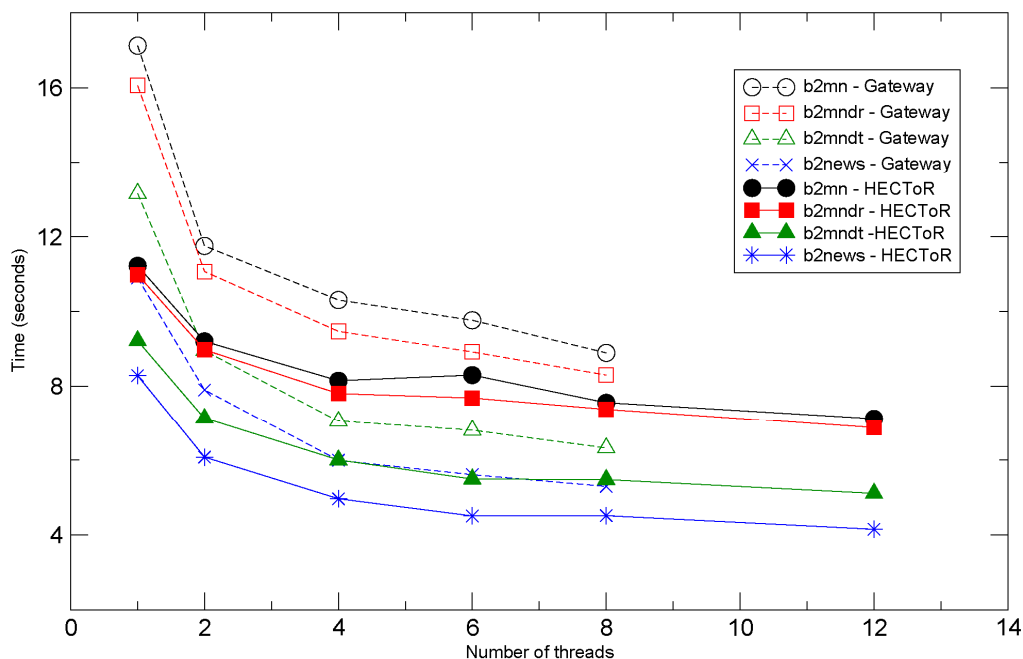
b2mn - main program runtime, this essentially times the entire code
 b2mndr - main driver routine for the code
 b2mndt - time stepping routine, b2news is called from within here
 b2news - performs one inner iteration of the b2 algorithm, essentially this is the core computation of the code

No. of threads	b2mn	b2mndr	b2mndt	b2news
Serial code	12.198	11.622	10.013	9.005
Serial code (back end)	11.444	11.115	9.331	8.363
1	11.219	10.983	9.214	8.281
2	9.198	8.972	7.126	6.078
4	8.144	7.797	6.002	4.965
6	8.295	7.677	5.491	4.504
8	7.551	7.372	5.477	4.512
12	7.120	6.873	5.109	4.149

Table 4: Showing performance of B2 running the AUG_16151_D+C+He benchmark on the EUFORIA BASE version of SOLPS5.0 after OpenMP parallelisation of all the species loops.

Examining the results in Table 4 we see that the serial code seems to take slightly longer than the single threaded OpenMP version. This is to be expected because the serial nodes on HECToR actually have slower processors than the batch nodes and therefore any performance comparisons should be made against either a serial version run on the backend on one processor or against a single threaded OpenMP version. We will make our performance comparisons against the single threaded OpenMP run. Figure 3 shows the performance of SOLPS5.0 plotted against number of threads for the AUG_16151_D+C+He benchmark. We have also included the results of running the code on the EUFORIA Gateway system. Because the AUG_16151_D+C+He benchmark contains only 12 species the maximum number of threads that can be used is 12.

Performance of the AUG_16151_D+C+He benchmark run for 10 time steps

**Figure 3: Showing the performance SOLPS5.0 for the AUG_16151_D+C+He benchmark.**

From Figure 3 we can see that the parallel version of the code gives a significant reduction in the b2news runtime...

From Figure 3 we can see that for the AUG_16151_D+C+He benchmark the performance of the b2news routine continues to improve up to 12 threads, although on HECToR there is little advantage in using more than 6 threads. On the Gateway system the b2news performance continues to improve on 8 threads (the thread limit on that system). On HECToR and the Gateway the fastest b2news time is obtained when running on 12 or 8 threads respectively. The fastest overall runtimes (b2mn) were also obtained for the largest thread counts. Comparing the performance against the single threaded OpenMP runtimes we observe a 37% improvement in the b2mn time and 50% improvement in the b2news time on HECToR. On the Gateway system the improvements are respectively 48% for b2mn and 51% b2news.

In addition to the 12 species AUG_16151_D+C+He benchmark we have also investigated the performance of a tungsten benchmark containing 92 species provided by Xavier Bonnin (XPB). This larger benchmark allows us utilise up to 24 threads on HECToR. This benchmark must be run with the EUFORIA BASE version of the code. In addition to using this version, the source file `include/DIMENSIONS.F` must be replaced with XPB's version and the B2 component of the code must be re-compiled from scratch. Full details of how to build and run the tungsten benchmark can be found in the SOLPS5.0 SVN under the path (login + password required for this):

https://solps-mdsplus.aug.ipp.mpg.de/wsvn/SOLPS/trunk/solps_examples/solps5.0/experimental/ITER-tungsten-xpb/README

The performance of the XPB benchmark on the HECToR XT6 and the Gateway system is given by Figure 4.

Performance of the XPB Tungsten benchmark for 25 time steps

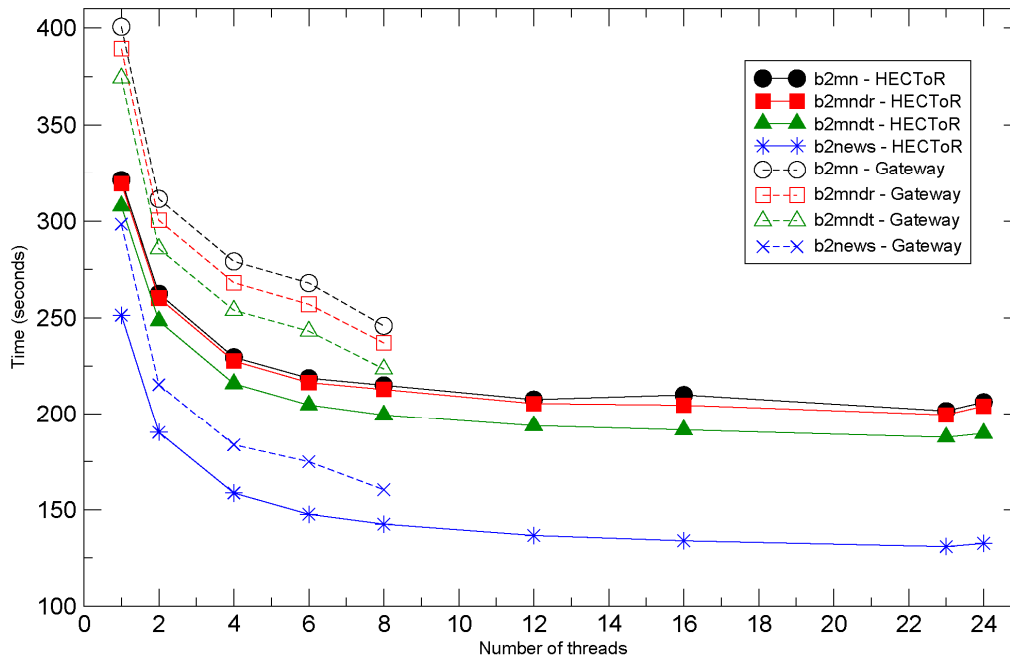


Figure 4: Showing the performance of XPB's 92 species tungsten benchmark on the HECToR XT6 system and the Gateway system.

The performance of the XPB benchmark is similar to that obtained for the 12 species AUG_16151_D+C+He benchmark. On HECToR using more than 8 threads gives little benefit, whereas on the Gateway the code still appears to be speeding up when using 8 threads. On HECToR we can investigate thread counts up to 24 and see that the best performance for both the `b2mn` and `b2news` times is obtained when using 23 threads, however, the difference in runtimes between the 8 thread case is minimal. The charging on the HECToR system is such that you pay for an entire compute node which gives access to 24 threads and thus even if only a small performance improvement is obtained then all 23 or 24 threads may as well be used. Comparing the runtimes against the one threaded OpenMP case on HECToR we see maximum improvements of 37% to the total runtime (`b2mn`) and 48% to the compute kernel (`b2news`) time. The Gateway achieved 39% improvement to the `b2mn` time and 46% improvement to the `b2news` time.

The XPB benchmark creates a significant amount of file I/O both in terms of output the ASCII `run.log` file and the number of binary output files. The total amount of output for a 25 time step run is just under 1 Gbyte. On HECToR I/O is known to be a limiting factor for many codes because the I/O resources are shared between all users of the system. If the I/O system is heavily loaded the runtime of an I/O intense code can be adversely affected. Previous work carried out under the EUFORIA project showed that changing the Lustre stripping parameters can help in some cases. However, in the case of SOLPS5.0, stripping across all available OST's had no effect on the code performance.

It should be noted that the parallelisation of the `nspecies` loops in SOLPS5.0 will always be limited by Amdahl's Law which states that speedup of a parallel program is limited by the time needed for the sequential fraction of the program. As `b2news` constitutes 85% of the total runtime of SOLPS5.0 this means that even if we were to perfectly parallelise all of the `b2news` routine a speedup of 3.90 on 8 threads would be the theoretical limit. As we have only parallelised the five `nspecies` loops

in `b2news` which constitute ~60% of the total runtime the maximum speed up we can expect to get on 8 threads is 2.1 which is indeed roughly what we have achieved.

The following observations were also made during the parallelisation process and may be of particular interest to the SOLPS developers. These observations relate to the AUG_16151_D+C+He benchmark but are expected to hold for the XPB benchmark too:

- Adding OpenMP directives to loop 4 had minimal impact on the performance of the `b2news` routine. As loop 4 contains a single line of code this is unsurprising. The overheads involved in creating and destroying the team of threads are likely to be equal to any saving obtained by using multiple threads.
- Adding OpenMP directives to loop 1 also had almost no impact on the performance of the code. This section of code wasn't called often enough for the benefits in parallelising it to outweigh any overhead
- Adding OpenMP directives to loop 3 did give us a small 6-11% improvement in the `b2news` runtime.
- Parallelising the continuity and momentum equations gives the biggest performance improvement with an almost 50% reduction in the `b2news` runtime. This performance enhancement is split equally between the continuity and momentum computation.

Conclusions

We have investigated the performance of the B2 component of the SOLPS5.0 fusion code. The `b2news` routine was identified to take over 85% of the total runtime and thus efforts were concentrated around speeding up this part of the code. An OpenMP loop based parallelisation of the loops over charge states (or species) has been carried out resulting in a performance improvement of up to 51% for the main compute kernel (`b2news`) and up to 48% for the total runtime of B2.

Based on the benchmarks tested in this study there appears to be little benefit of using more than 8 threads on the HECToR system. The shared I/O resources on that system are likely to cause the performance tail off beyond 8 threads. On the Gateway system the code is continuing to scale to 8 threads and thus it's possible that using a system with additional cores may still be beneficial.

References

- [1] SOLPS Wiki page
http://solps-mdsplus.aug.ipp.mpg.de/wiki/SOLPS/index.php/Main_Page
- [2] SOLPS 5.0 Online Manual, April 2010.
<http://www.ipp.mpg.de/~dpc/solps/solps.pdf>
- [3] SOLPS Wiki page describing how to compile and run SOLPS on the TOK Linux cluster in Garching.
[Compiling and running SOLPS on the TOK](#)

Appendix

ÅBO version compiler.* files

The full listing of `src/Eirene/config/compiler.linux.pgi.xt6` for the ÅBO version of SOLPS 5.0 is:

```
DEFINES = ${SOLPS_DEFINES} ${EIRENE_DEFINES} -DRECL_UNITS_PER_DOUBLE=8
          -DNO_FORT10 -DNO_SHORT_CYCLE -DNO_SAVE

CPP      = -/lib/cpp -traditional
FC       = ftn

ifdef SOLPS_DEBUG
FFLAGS   = -g -C
else
FFLAGS   = -O3
endif

COMPILE   = ${FC} -c ${FFLAGS}

LDLIBS    = -L$(SOLPSTOP)/src/lib/${OBJECTCODE}
LDEXTRA    =
INCMOD    = -I
MOD        = mod
```

The listing for `src/Braams/b2/base/config/compiler.linux.pgi.xt6` for the ÅBO version of the code is:

```
EQUIVS = -Dsdot=ddot -Dsasum=dasum -Dsaxpy=daxpy -Dsscal=dscal
          -Dsnrm2=dnrm2 -Dscopy=dcopy -Dsgbtrf=dgbtrf -Dsgbtrs=dgbtrs
          -DGRREAL=4 -DDOFLUSH -DNO_NAG

CPP = cpp -C -traditional
CPOPTS = -C -P -I${srcdir} ${EQUIVS}
SED = sed s:\#PREC\#:d:g
SED =
LINK_MOD =

FC = ftn

ifdef SOLPS_DEBUG
FCOPTS = -g -C
else
FCOPTS = -O3
endif

DBLFC=${FC}
DDBLOPTION=-Ad # ???
DBLSED = sed -e s:\#PREC\#:d:g
DBLSED =

INCMOD = -I
MOD = mod
OBJDEST = -o ${OBJECTCODE}/${*.o}

BLD = ar rcv

GRLIBES = ${NCARG} -lpgm -lmgmath
LDLIBES = ${NAG} -lnetcdf

ifdef USE_EIRENE
LDLIBSEXTRA = ${OBJECTCODE}/libeirene.a ${OBJECTCODE}/grlib.o
endif

LD = $(FC)

LCPPE=-L/afs/ipp-
garching.mpg.de/u/dpc/solps/solps4.0_test/src/lib/${OBJECTCODE}
-lsonnet /usr/lib/libstdc++.so.2.9 -lblas
```

EUFORIA BASE version compiler.* files

The full listing of `src/Eirene/config/compiler.linux.pgi.xt6` for the EUFORIA BASE version of SOLPS 5.0 is:

```
DEFINES          = ${EIRENE_DEFINES} ${SOLPS_CPP} -DRECL_UNITS_PER_DOUBLE=8

CPP              = -/lib/cpp -traditional
FC               = ftn

ifdef SOLPS_DEBUG
FFLAGS          = -g -C
else
FFLAGS          = -O3
endif

COMPILE          = ${FC} -c ${FFLAGS}

GKS              = -lX11 -lXt

LDLIBS = -L$(SOLPSTOP)/src/lib/${OBJECTCODE}
LDEXTRA          =

INCMOD = -I
MOD = mod
```

The listing for `src/Braams/b2/base/config/compiler.linux.pgi.xt6` for the EUFORIA BASE version of the code is:

```
EQUIVS = -Dsdot=ddot -Dsasum=dasum -Dsaxpy=daxpy -Dscal=dscal -Dsnrm2=dnrm2
        -Dscopy=dcopy -Dsgbtrf=dgbtrf -Dsgbtrs=dgbtrs -Dsgetri=dgetri
        -Dsgetrf=dgetrf -DPERF_TIMER

CPP = cpp -C -traditional
CPOPTS = -C -P -I${srcdir} ${EQUIVS}
SED = sed s:\#PREC\#:d:g
SED =
LINK_MOD =

FC = ftn

ifdef SOLPS_DEBUG
FCOPTS = -g -C
else
FCOPTS = -O3
endif

DBLFC=${FC}
DDBLOPTION=-Ad # ???
DBLSED = sed -e s:\#PREC\#:d:g
DBLSED =

# -I is used to specify search path for modules
INCMOD = -I
MOD = mod
OBJDEST = -o ${OBJECTCODE}/${*.o}

BLD = ar rcv

GRLIBES = ${NCARG} -lcm -lncmath
LDLIBES = ${NAG} -lnetcdf

ifdef USE_EIRENE
LDLIBSEXTTRA = ${OBJECTCODE}/libeirene.a ${OBJECTCODE}/grlib.o
endif

LD = $(FC)
#Use this line for the debug version
#LDOPTS = -g -C
LDOPTS =
```



```
LCP--L/afs/ipp-  
garching.mpg.de/u/dpc/solps/solps4.0_test/src/lib/${OBJECTCODE} -lsonnet  
/usr/lib/libstdc++.so.2.9 -lblas
```

Summary of changes made to the SOLPS5.0 source files

This section summarises the main changes made to the SOLPS source files.

New files added:

modules/b2mod_init_openmp.F – new module which initialises OpenMP
modules/b2mod_timer.F – new module which allocates timer arrays
utility/perf_fiona.F – new routine used to time SOLPS routines, uses MPI routine `MPI_Wtime()` for the timings but could easily be altered to use `OMP_GET_WTIME` instead.

Files altered:

sources/b2sigp.F

- `ncall` declared `THREADPRIVATE`

sources/b2sifr.F

- Variables `ncall`, `styl0`, `styl1`, `lambda`, `phm0`, `phm1`, `phm2`, `limthii`, `limthee` declared as `THREADPRIVATE`
- Parenthesis moved to fix bug highlighted by XPB.

driver/b2mn.F

- Pre-processor text added to ensure OpenMP version warns that output may not be as expected.

transport/b2tfnb.F

- Module `b2mod_init_openmp` is added to allow the use of `OMP_IN_PARALLEL`
- All save variables declared as `THREADPRIVATE` – `ncall` now local to each thread
- Pre-processor flags and `OMP_IN_PARALLEL` have been added around the `ncall` related I/O statement to indicate that OpenMP is being used.

solvers/b2usco.F

- Module `b2mod_init_openmp` is added to allow the use of `OMP_IN_PARALLEL`
- `ncall` declared `THREADPRIVATE`
- Output flagged via pre-processor flags and `OMP_IN_PARALLEL`

solvers/b2ursc.F

- `ncall` declared `THREADPRIVATE`

solvers/b2ux5p.F

- Module `b2mod_init_openmp` is added to allow the use of `OMP_IN_PARALLEL`
- Variables `iwrk`, `aaf`, `wrk0`, `wrk1`, `wrk2`, `ncall`, `style`, `acpar`, `cpu`
- Conditionals added around the solvers to ensure the code will stop and report an error if the `iluter` or `slv5pt` solvers are used
- Output flagged via pre-processor flags and `OMP_IN_PARALLEL`

solvers/b2urmo.F

- `ncall` declared `THREADPRIVATE`

solvers/b2usmo.F

- `ncall`, `cfc0` declared `THREADPRIVATE`

solvers/b2upco.F

- `ncall` declared `THREADPRIVATE`

utility/ilutern_solps.F

- Module b2mod_init_openmp is added to allow the use of OMP_IN_PARALLEL
- lfirst, ncall, nwrite, /ma28fd/, /ma28hd/ declared THREADPRIVATE
- Output flagged via pre-processor flags and OMP_IN_PARALLEL. The output file b2_matrix will not be written out when running in parallel.

utility/ipmain.F

- Increased the size of ipgtmx from 20 to 100 to allow more than 20 threads to run at once.

utility/subsys.F

- Module b2mod_init_openmp is added to allow the use of OMP_IN_PARALLEL
- Module b2mod_timer added to allow the use of the timing routines in perf_fiona.F
- Pre-processor flags added for timer calls and also to ensure timer is only called on thread 0 when running in parallel.
- Added early return to stack tracing when running in parallel.

utility/sysend.F

- Pre-processor flags added for timer calls
- Call to perfout_all added to output timer results.

utility/ma28.F

- All common blocks declared as THREADPRIVATE

modules/b2mod_ma28.F

- All variables declared as THREADPRIVATE

equations/b2npmo.F

- ncall, rxg, vis_par, diagno, solvedum declared THREADPRIVATE
- PARALLEL DO added around main nspecies loop

equations/b2news.F

- Four PARALLEL DO blocks added around each of the nspecies loops

equations/b2nxst.F

- ncall declared THREADPRIVATE

equations/b2nxfc.F

- ncall declared THREADPRIVATE

equations/b2npco.F

- ncall, rxg, pcm0, pcm1 declared THREADPRIVATE

equations/b2nxdv.F

- ncall declared THREADPRIVATE